Podstawy programowania w Java

mgr inż. Sławomir Aleksander Dańczak

Katedra Metod Programowania
Polsko-Japońska Akademia Technik Komputerowych



JAVA

Operacje, instrukcje, zmienne

Wykład 02

Plan na dzisiaj...

00

OPERACJE → OPERACJE BITOWE → INSTRUKCJE → ZMIENNE

Zapisy literałów

-01

Reprezentacja wartości w kodzie

LICZBY → ZNAKI → LOGIKA → WARTOŚCI

Zapisy literałów

W języku Java przez literał rozumiany jest reprezentowana w kodzie źródłowym wartość:

- typu prostego (primitive type),
- typu złożonego **String**,
- lub typu **null** języka Java

Literały typów prostych

Literały stanu logicznego:

- true
- false

Literały typów prostych

Literały liczb całkowitych w zapisie:

- dzisiętnym
- binarnym
- ósemkowym
- szesnastkowym

Literały typów prostych

Literały liczb całkowitych w **zapisie dziesiętnym:**

- ciąg cyfr rozpoczynający się wartością inną niż **0**
- opcjonalnie występująca końcówka **l L**

Literały typów prostych

Literały liczb całkowitych w **zapisie ósemkowym:**

- ciąg cyfr rozpoczynający się wartością **0**
- opcjonalnie występująca końcówka **l L**

Literały typów prostych

Literały liczb całkowitych w **zapisie szesnastkowym:**

- ciąg cyfr rozpoczynający się wartością **0x**
- opcjonalnie występująca końcówka **l L**

Literały typów prostych

Literały liczb całkowitych w **zapisie binarnym:**

- ciąg cyfr rozpoczynający się wartością **0b**
- opcjonalnie występująca końcówka **l L**

Literały typów prostych

Literały znakowe:

- pojedynczy znak objęty apostrofem np: **'a'**, **'A'** lub **'0'**
- objęty apostrofem znak ukośnik (backslash) i następujący po nim znak specjalny lub kod znaku np: **'\t'**, **'\177'**
- objęty apostrofem znak ukośnik (backslash), następujący po niem znak u i kod znaku reprezentowany w UTF-16 np: **'\u03a9'**, **'\uFFFF'**

Literały typów prostych

Literały liczb zmiennoprzecinkowych:

- 2.f
- .3f
- 3.14
- 1e-9d
- 1e127

Operatory

01

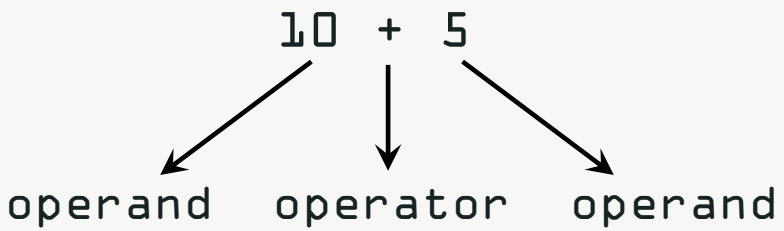
Wykonywanie operacji na wartościach

`+` → `-` → `*` → `/` → `%`

Operatory

Czym jest operator?

Operator to znak lub symbol określający operację wykonywaną na danych.



Operator to znak lub symbol, który służy do wykonywania **określonej operacji** na **jednym lub więcej operandach**, w celu uzyskania określonego **wyniku**.

Operatory

Jakie operacje?

Operatory reprezentują:

- operacje arytmetyczne`
- operacje logiczne
- porównania
- przypisania
- ...

Operatory logiczne

! zaprzeczenie

!	false	true
	true	false

&& koniunkcja

&&	false	true
false	false	false
true	false	true

|| alternatywa

||	false	true
false	false	true
true	true	true

Obowiązujące zasady:

- argumenty i rezultaty operacji orzecznikowych są typu **boolean**

Operatory arytmetyczne

`+`	dodawanie	`-`	odejmowanie
`*`	mnożenie	`/`	dzielenie
`%`	reszta z dzielenia		
`++`	zwiększenie o 1	`--`	zmniejszenie o 1
`+=`	dodanie	`-=`	odjęcie
`*=`	pomnożenie przez	`/=`	podzielenie przez

Operatory porównania

==	równe	!=	różne
<	mniejsze	>	większe
<=	mniejsze lub równe	>=	większe lub równe

Operatory prefiksowe i postfiksowe

++value	inkrementacja prefiksowa	**value++**	inkrementacja postfiksowa
--value	dekrementacja prefiksowa	**value--**	dekrementacja postfiksowa

Operacja **prefiksowa** modyfikuje argument, zwiększając albo zmniejszając go o 1, a w miejscu wykonania dostarcza **nową** wartość.

Operacja **postfiksowa** modyfikuje argument, zwiększając albo zmniejszając go o 1, a w miejscu wykonania dostarcza **starą** wartość.

Wyrażenia

Wyrażenia są zestawami **operacji**.

Do wykonywania operacji na zmiennych służą **operatory**.

W celu określenia kolejności w jakiej mają być wykonywane operacje wykorzystuje się **nawiasy** oraz **priorytet** i **wiązanie** operatora.

Priorytety operatorów

Priorytet	Operator	Opis	Kierunek wiązania
1	()	Nawiasy (określenie kolejności)	Od lewej do prawej
1	[], .	Dostęp do tablicy, pola, wywołanie metody	Od lewej do prawej
2	val++, val--	Postinkrementacja i postdekrementacja	Od lewej do prawej
3	++val,--val, +, -, ~, !	Preinkrementacja, predekrementacja, operatory jednoargumentowe	Od prawej do lewej
3	(type)	Konwersja typu	Od prawej do lewej
3	new	Tworzenie nowego obiektu	Od prawej do lewej
4	*, /, %	Mnożenie, dzielenie, reszta z dzielenia	Od lewej do prawej
5	+, -	Dodawanie, odejmowanie	Od lewej do prawej
6	<<, >>, >>>	Operatory przesunięcia bitowego	Od lewej do prawej
7	<=, >, >=	Operatory porównania	Od lewej do prawej
7	instanceof	Sprawdzenie typu	Od lewej do prawej
8	==, !=	Równość, nierówność	Od lewej do prawej
9	&	Bitowy AND	Od lewej do prawej
10	^	Bitowy XOR	Od lewej do prawej
11	|	Bitowy OR	Od lewej do prawej
12	&&	Logiczny AND	Od lewej do prawej
13	||	Logiczny OR	Od lewej do prawej
14	? :	Operator trójargumentowy (warunkowy)	Od prawej do lewej
15	=, +=, -=, *=, /=, %=, &=, |=, =^, <<=, >>=, >>>=	Operatory przypisania	Od prawej do lewej

Wyrażenia

Wyrażenia są zestawami **operacji**.

Do wykonywania operacji na zmiennych służą **operatory**.

W celu określenia kolejności w jakiej mają być wykonywane operacje wykorzystuje się **nawiasy** oraz **priorytet** i **wiązanie** operatora.

Uwzględnianie nawiasów:

```
a + (b - c)
```

operacja dodawania zostanie wykonana dopiero po wykonaniu operacji odejmowania.

Uwzględnienie priorytetów:

Jeśli operacją są o **różnych priorytetach**, to operację wyrażoną przez operator o niższym priorytecie wykonuje się po operacji o wyższym priorytecie.

Wyrażenia

Uwzględnienie wiązań:

Jeśli operacje mają równy priorytet, to kolejność operacji określona jest przez **wiązanie.**

Jeśli wiązanie operacji jest lewe, to najpierw wykonuje się operację wyrażoną przez operator znajdujący się **z lewej strony wyrażenia**

Jeśli wiązanie operacji jest prawe, to najpierw wykonuje się operację wyrażoną przez operator znajdujący **się z prawej strony wyrażenia.**

Wyrażenia

Ponieważ wiązanie operatorów dodawania i odejmowania są lewe więc wyrażenie

```
a - b + c
```

zostanie potraktowane jako wyrażenie

```
(a - b) + c
```

Ponieważ wiązanie operatora przypisania jest prawe więc wyrażenie

```
a = b = c
```

zostanie potraktowane jako wyrażenie

```
a = (b = c)
```

Operatory bitowe

02

Wykonywanie operacji
bezpośrednio na bitach wartości

AND → OR → XOR → NOT

Operatory bitowe

Operatory bitowe są pozwalają wykonywać operacje na bitach

Operacje bitowe są zdefiniowane tylko dla typów całkowitych.

Dysponujemy następującymi operatorami:

~	NOT
&	AND
|	OR
^	XOR
<<	left shift
>>	signed right shift
>>>	unsigned right shift

Operatory bitowe

~ NIE (NOT)

A	$\sim A$
0_2	1_2
1_2	0_2

| LUB (OR)

A	B	$A \mid B$
0_2	0_2	0_2
0_2	1_2	1_2
1_2	0_2	1_2
1_2	1_2	1_2

& I (AND)

A	B	$A \,\&\, B$
0_2	0_2	0_2
0_2	1_2	0_2
1_2	0_2	0_2
1_2	1_2	1_2

^ ALBO (XOR)

A	B	$A \wedge B$
0_2	0_2	0_2
0_2	1_2	1_2
1_2	0_2	1_2
1_2	1_2	0_2

Operatory bitowe ~ NIE (NOT)

Na przykładzie typu danych

byte (8b)

A	$\sim A$
0_2	1_2
1_2	0_2

$\sim 0_{10} = ?$

$0_{10} = 0000\ 0000_2$

$\frac{\sim 0000\ 0000_2}{1111\ 1111_2}$

$1111\ 1111_2 = -1_{10}$

$\sim 0_{10} = -1_{10}$

$\sim 1_{10} = ?$

$1_{10} = 0000\ 0001_2$

$\frac{\sim 0000\ 0001_2}{1111\ 1110_2}$

$1111\ 1110_2 = -2_{10}$

$\sim 1_{10} = -2_{10}$

$\sim -52_{10} = ?$

$-52_{10} = 1100\ 1100_2$

$\frac{\sim 1100\ 1100_2}{0011\ 0011_2}$

$0011\ 0011_2 = 51_{10}$

$\sim -52_{10} = 51_{10}$

Operatory bitowe & I (AND)

Na przykładzie typu danych

byte (Bb)

A	B	$A \,\&\, B$
0_2	0_2	0_2
0_2	1_2	0_2
1_2	0_2	0_2
1_2	1_2	1_2

$53_{10} \,\&\, 16_{10} = ?$

$53_{10} = 0011\ 0101_2$

$16_{10} = 0001\ 0000_2$

$$\begin{array}{r} 0011\ 0101_2 \\ \&\ 0001\ 0000_2 \\ \hline 0001\ 0000_2 \end{array}$$

$0001\ 0000_2 = 16_{10}$

$53_{10} \,\&\, 16_{10} = 16_{10}$

$-18_{10} \,\&\, 23_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$23_{10} = 0001\ 0111_2$

$$\begin{array}{r} 1110\ 1110_2 \\ \&\ 0001\ 0111_2 \\ \hline 0000\ 0110_2 \end{array}$$

$0000\ 0110_2 = 6_{10}$

$-18_{10} \,\&\, 23_{10} = 6_{10}$

$120_{10} \,\&\, 12_{10} = ?$

$120_{10} = 0111\ 1000_2$

$12_{10} = 0000\ 1100_2$

$$\begin{array}{r} 0111\ 1000_2 \\ \&\ 0000\ 1100_2 \\ \hline 0000\ 1000_2 \end{array}$$

$0000\ 1000_2 = 8_{10}$

$120_{10} \,\&\, 12_{10} = 8_{10}$

Operatory bitowe | LUB (OR)

Na przykładzie typu danych

byte (Bb)

A	B	$A \mid B$
0_2	0_2	0_2
0_2	1_2	1_2
1_2	0_2	1_2
1_2	1_2	1_2

$53_{10} \mid 16_{10} = ?$

$53_{10} = 0011\ 0101_2$

$16_{10} = 0001\ 0000_2$

$$\begin{array}{r} 0011\ 0101_2 \\ \underline{\mid 0001\ 0000_2} \\ 0011\ 0101_2 \end{array}$$

$0011\ 0101_2 = 53_{10}$

$53_{10} \mid 16_{10} = 53_{10}$

$-18_{10} \mid 23_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$23_{10} = 0001\ 0111_2$

$$\begin{array}{r} 1110\ 1110_2 \\ \underline{\mid 0001\ 0111_2} \\ 1111\ 1111_2 \end{array}$$

$1111\ 1111_2 = -1_{10}$

$-18_{10} \mid 23_{10} = -1_{10}$

$120_{10} \mid 12_{10} = ?$

$120_{10} = 0111\ 1000_2$

$12_{10} = 0000\ 1100_2$

$$\begin{array}{r} 0111\ 1000_2 \\ \underline{\mid 0000\ 1100_2} \\ 0111\ 1100_2 \end{array}$$

$0111\ 1100_2 = 124_{10}$

$120_{10} \mid 12_{10} = 124_{10}$

Operatory bitowe ^ ALBO (XOR)

Na przykładzie typu danych

byte (8b)

A	B	$A \wedge B$
0_2	0_2	0_2
0_2	1_2	1_2
1_2	0_2	1_2
1_2	1_2	0_2

$53_{10} \wedge 16_{10} = ?$

$53_{10} = 0011\ 0101_2$

$16_{10} = 0001\ 0000_2$

$$\begin{array}{r} 0011\ 0101_2 \\ \wedge\ 0001\ 0000_2 \\ \hline 0010\ 0101_2 \end{array}$$

$0010\ 0101_2 = 37_{10}$

$53_{10} \wedge 16_{10} = 37_{10}$

$-18_{10} \wedge 23_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$23_{10} = 0001\ 0111_2$

$$\begin{array}{r} 1110\ 1110_2 \\ \wedge\ 0001\ 0111_2 \\ \hline 1111\ 1001_2 \end{array}$$

$1111\ 1001_2 = -7_{10}$

$-18_{10} \wedge 23_{10} = -7_{10}$

$120_{10} \wedge 12_{10} = ?$

$120_{10} = 0111\ 1000_2$

$12_{10} = 0000\ 1100_2$

$$\begin{array}{r} 0111\ 1000_2 \\ \wedge\ 0000\ 1100_2 \\ \hline 0111\ 0100_2 \end{array}$$

$0111\ 0100_2 = 116_{10}$

$120_{10} \wedge 12_{10} = 116_{10}$

Operatory przesunięć bitowych

Operatory przesunięć bitowych służą do przesuwania bitów w lewo lub w prawo.

W przypadku podstawowych operatorów przesunięć bitowych rozróżniamy:

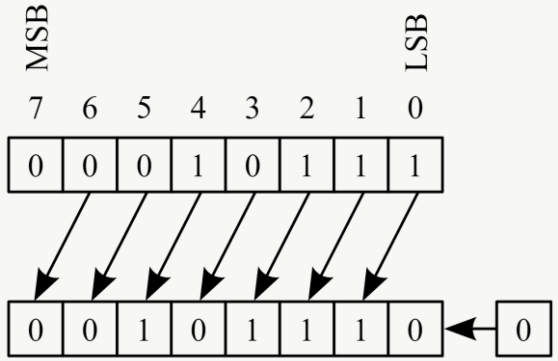
- Przesunięcie arytmetyczne:
  - << (left shift)
  - >> (signed right shift)
- Przesunięcie logiczne:
  - >>> (unsigned right shift)

Operatory bitowe

<< (left shift)

Na przykładzie typu danych

byte (8b)



$53_{10} \ll 3_{10} = ?$

$53_{10} = 0011\ 0101_2$

$0011\ 0101_2 \ll 3_{10}$

$001|1010\ 1000_2$

$1010\ 1000_2$

$1010\ 1000_2 = -88_{10}$

$53_{10} \ll 3_{10} = -88_{10}$

$-18_{10} \ll 2_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$1110\ 1110_2 \ll 2_{10}$

$11|1011\ 1000_2$

$1011\ 1000_2$

$1011\ 1000_2 = -72_{10}$

$-18_{10} \ll 2_{10} = -72_{10}$

$120_{10} \ll 10_{10} = ?$

$120_{10} = 0111\ 1000_2$

$0111\ 1000_2 \ll 10_{10}$

$01\ 1110\ 0000|0000\ 0000_2$

$0000\ 0000_2$

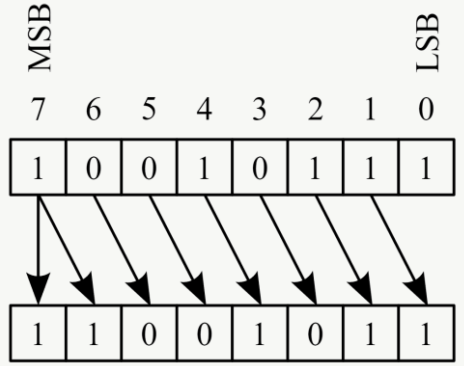
$0000\ 0000_2 = 0_{10}$

$120_{10} \ll 10_{10} = 0_{10}$

Operatory bitowe >> (signed right shift)

Na przykładzie typu danych

byte (8b)



$53_{10} \gg 3_{10} = ?$

$53_{10} = 0011\ 0101_2$

$0011\ 0101_2 \gg 3_{10}$

$0000\ 0110|101_2$

$0000\ 0110_2$

$0000\ 0110_2 = 6_{10}$

$53_{10} \gg 3_{10} = 6_{10}$

$-18_{10} \gg 2_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$1110\ 1110_2 \gg 2_{10}$

$1111\ 1011|10_2$

$1111\ 1011_2$

$1111\ 1011_2 = -5_{10}$

$-18_{10} \gg 2_{10} = -5_{10}$

$120_{10} \gg 10_{10} = ?$

$120_{10} = 0111\ 1000_2$

$0111\ 1000_2 \gg 10_{10}$

$0000\ 0000\ |\ 0001\ 1110\ 00_2$

$0000\ 0000_2$

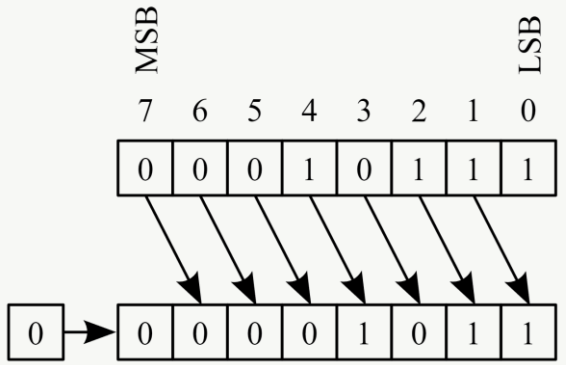
$0000\ 0000_2 = 0_{10}$

$120_{10} \gg 10_{10} = 0_{10}$

Operatory bitowe >>> (unsigned right shift)

Na przykładzie typu danych

byte (8b)



$53_{10} \ggg 3_{10} = ?$

$53_{10} = 0011\ 0101_2$

$0011\ 0101_2 \ggg 3_{10}$

$0000\ 0110|101_2$

$0000\ 0110_2$

$0000\ 0110_2 = 6_{10}$

$53_{10} \ggg 3_{10} = 6_{10}$

$-18_{10} \ggg 2_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$1110\ 1110_2 \ggg 2_{10}$

$0011\ 1011|10_2$

$0011\ 1011_2$

$0011\ 1011_2 = 59_{10}$

$-18_{10} \ggg 2_{10} = 59_{10}$

$120_{10} \ggg 10_{10} = ?$

$120_{10} = 0111\ 1000_2$

$0111\ 1000_2 \ggg 10_{10}$

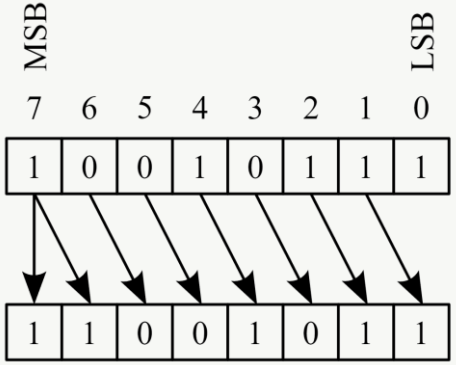
$0000\ 0000 | 0001\ 1110\ 00_2$

$0000\ 0000_2$

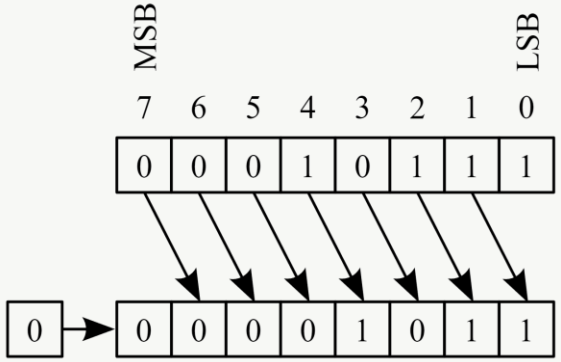
$0000\ 0000_2 = 0_{10}$

$120_{10} \ggg 10_{10} = 0_{10}$

Operatory bitowe



Różnica?



$-18_{10} \gg 2_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$1110\ 1110_2 \gg 2_{10}$

$1111\ 1011|10_2$

$1111\ 1011_2$

$1111\ 1011_2 = -5_{10}$

$-18_{10} \gg 2_{10} = -5_{10}$

$-18_{10} \ggg 2_{10} = ?$

$-18_{10} = 1110\ 1110_2$

$1110\ 1110_2 \ggg 2_{10}$

$0011\ 1011|10_2$

$0011\ 1011_2$

$0011\ 1011_2 = 59_{10}$

$-18_{10} \ggg 2_{10} = 59_{10}$

Instrukcje

03

Polecenia określające działanie programu

POLECENIE → PORZĄDEK → WYKONANIE → RESULTAT

Instrukcje

Czym jest instrukcja?

Instrukcja jest poleceniem wydanym procesorowi komputera, przez program

Instrukcja to podstawowa jednostka działania w programach komputerowych. Mówimy o nich w kontekście przekazywania poleceń do wykonania w określonym porządku. Mogą one zawierać konkretne operacje, warunki, pętle i wiele innych elementów, które wpływają na zachowanie programu.

Instrukcje

Instrukcje dzielą się na **czynne** i **bierne**.

Instrukcją **bierną** jest

- instrukcja **deklaracyjna**

Natomiast instrukcjami **czynnymi** są:

- instrukcja **pusta**
- instrukcja **grupująca**
- instrukcja **wyrażeniowa**
- instrukcja **warunkowa**
- instrukcje **iteracyjne** i **decyzyjne**
- instrukcje **zaniechania** i **kontynuowania**

Każda instrukcja jest zakończona klamrą albo średnikiem.

Instrukcja pusta (czynna)

Instrukcja pusta ma postać:

```
;
```

Instrukcja grupująca (czynna)

Instrukcja grupująca ma postać

```
{
    Instrukcja1;
    Instrukcja2;
    ...
    InstrukcjaN;
}
```

Przykład:

```
{
    System.out.println("Hello_old_friend");
}
```

Instrukcja wyrażeniowa (czynna)

Instrukcja wyrażeniowa ma postać:

exp;

w której exp jest: przypisaniem, wywołaniem, inkrementacją albo dekrementacją wartości

Instrukcja deklaracyjna (bierna)

Instrukcja:

Typ nazwa;

deklaruje zmienną **nazwa** typu **Typ**.

Np. instrukcja:

int age = 7;

deklaruje zmienną **age** typu **int** (całkowitego), którą podczas deklarowania zainicjowano wartością **7** reprezentowaną przez literał.

Zmienne

04

Przechowywanie i wykorzystywanie danych w programie

TYP → NAZWA → WARTOŚĆ → PAMIĘĆ

Zmienne

Czym jest zmienna?

Zmienną jest obszar **pamięci operacyjnej**, w której przechowywane są dane.

Od **typu** zmiennej zależy:

- **rozmiar** zajmowanego obszaru
- **zakres** wartości danych jakie można przypisywać zmiennej
- **reprezentacja** danych

Rodzaje zmiennych

Zmienne można podzielić na:

- pierwotne
- odnośnikowe

Zmienne typów pierwotnych (prostych)

- logiczne
- numeryczne
  - całkowite
  - rzeczywiste
  - znakowe

Zmienne typów odnośnikowych (złożonych)

- tablice
- obiekty

Typ logiczny

Typ	*Rozmiar*	*Zakres*
boolean	*???*	true/false

Java Language Specification nie definiuje konkretnego rozmiaru boolean w pamięci; zależy on od implementacji JVM.

Typ numeryczny - Typ całkowitoliczbowy

Typ	*Rozmiar*	*Zakres*
byte	1 bajt	-128 do 127
short	2 bajty	-32 768 do 32 767
int	**4 bajty**	**-2 147 483 648 do 2 147 483 647**
long	8 bajtów	-9 223 372 036 854 775 808 do 9 223 372 036 854 775 807

```
10 - system dziesiętny
10L
10l
0b1010 - zapis w systemie binarnym (dwójkowym)
010 - zapis w systemie oktalnym (ósemkowym)
0x5A - zapis w systemie heksadecymalnym (szesnastkowym)
```

Typ numeryczny - Typ zmiennoprzecinkowy

Typ	*Rozmiar*	*Zakres*
float	4 bajty	ok. $\pm 3{,}4 * 10^{38}$, ~6-7 cyfr znaczących
double	**8 bajtów**	**ok. $\pm 1{,}8 * 10^{308}$, ~15-16 cyfr znaczących**

```
3.14
3.14d
3.14D
3.14f
3.14F
```

Typ numeryczny - Typ znakowy

Typ	*Rozmiar*	*Zakres*
char	2 bajty	0 do 65 535 (\u0000-\uFFFF)

’a’

Zadania wykładowe

05

01 Napisz program w Javie, który utworzy dwie zmienne całkowite o wartościach 15 i 4. Oblicz i wyświetl ich sumę, różnicę, iloczyn, wynik dzielenia oraz resztę z dzielenia.

02 Utwórz dwie zmienne całkowite a = 12 oraz b = 5. Bez uruchamiania programu określ wyniki poniższych wyrażeń, uwzględniając priorytety operatorów. Następnie sprawdź swoje odpowiedzi w Javie.

- **a + b * 2**
- **(a + b) * 2**
- **a % b + 3**
- **a > b && b > 0**

03 Utwórz zmienną całkowitą o wartości 42. Wyświetl jej wartość, a następnie wyniki zastosowania operatorów bitowych &, | oraz ^ z liczbą 15. Dla każdego działania zapisz obie liczby również w postaci binarnej i spróbuj przewidzieć wynik przed uruchomieniem programu.

Pytania?

06

> waiting for input...

Dziękuję za uwagę

07

THE END